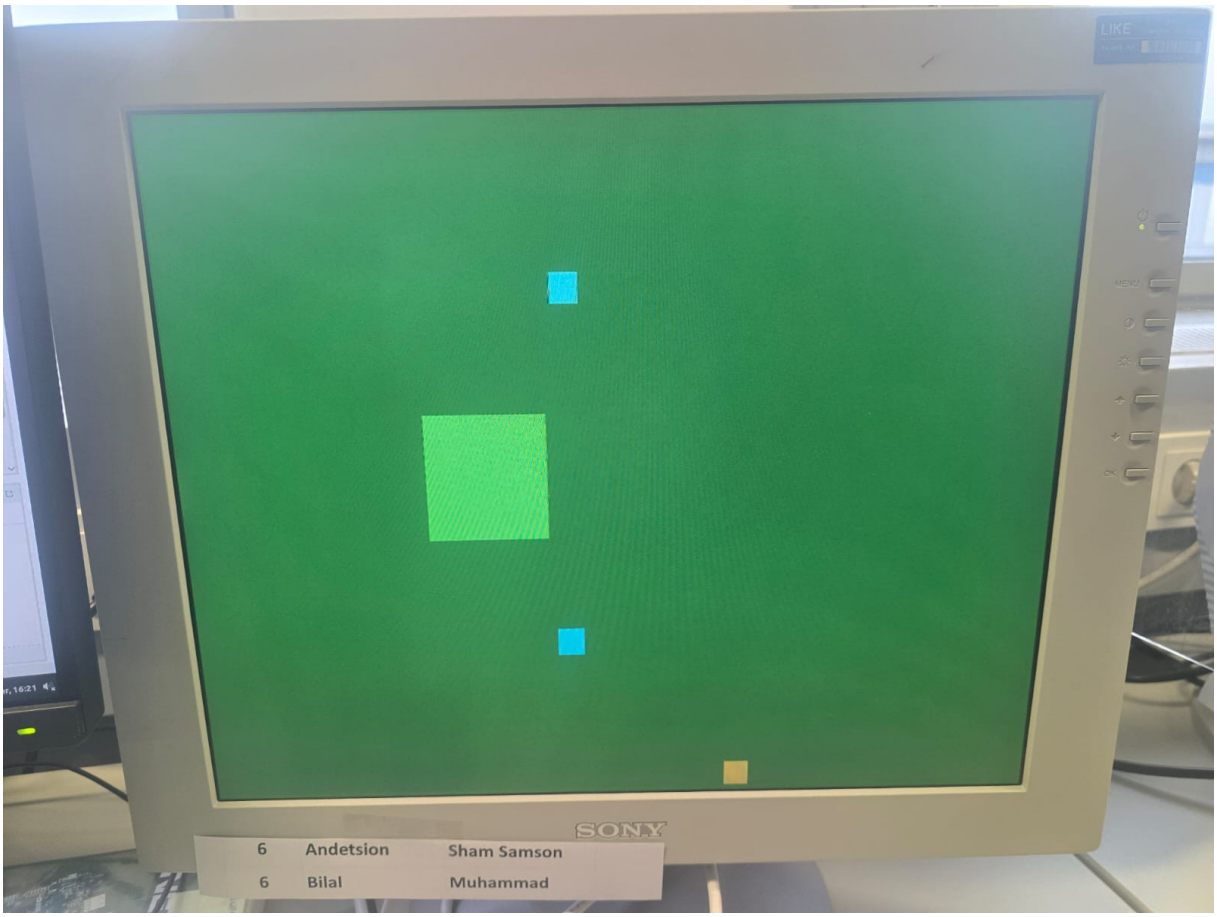
Friedrich-Alexander-Universität
Technische Fakultät

Hungry Snake

Muhammad Bilal, Sham Samson Andetsion



PrASIC Febr. 25

WS 2024/2025

Lecturer/Chair

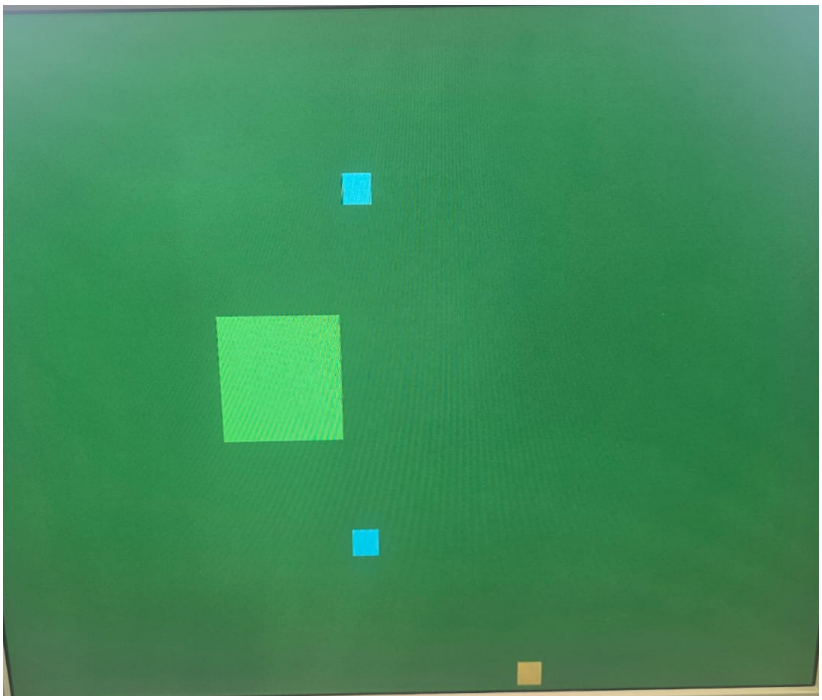
Prof. Dr.-Ing. Jürgen Frickel

24.March 2025

Hungry Snake

1.1 Introduction

The name of the game is Hungry Snake. The game is just like the old typical snake game we used to play during our childhood. The game follows the typical logic of the snake game. The food is generated randomly on the screen and when snake eats the food its size increases. Instead of increasing the length of the snake in our logic the width and the length of the snake increases. Whenever snake hits the border of the screen it dies, and we also created two blocks on the screen so when snake hit those blocks it also dies. This is the whole idea of the game. The below picture shows the outlook of the game.



In the picture, the green block is the snake, the yellow block is the food, and two blue blocks are the rocks.

1.2 Idea

We have 2 ideas for the game first one is the snake game, and the other one was the space invaders. We decide to go with the snake game as we both have experience with the game during the earlier days of life.

1.3 Structure of the game

The overall game structure consists of 5 modules and 1 top module. The first module is the VGA module to connect and displaying content on the VGA screen. The other modules are the snake module, food module, the block module and last module to display score on 7 segment displays on the board.

1.3.1 Top Module:

In the top module, we initialize the all the Nexys 4 board resources which we are going to use in our game and, initialize the other module and perform operations required instances from different modules.

In HungrySnake module (Top Module), other than initialization we have performed following function which we are used in different modules of the game. The functions which are implements in the top module are:

- Drawing of the snake on the VGA screen and the background.
- Collision detection between the snake and the food.
- Collision detection between snake and the blocks.
- Game Over scenarios and display.
- Score counter

1.3.2 VGA Module

This is the basic module which used to program the VGA screen. In this module, we set the value for the horizontal pixel and the vertical pixel. This is one of the most important modules of the game. This module correctly generated synchronization signals, managed pixel positioning, and controlled frame timing. The system displayed graphics on the VGA screen without issues.

1.3.3 Snake module

As the name indicates in this module, we do all the necessary things related to the snake. At first, we set the initial parameters of the snake which includes its intial size in both horizontal and vertical dimension along with the location of the snake when the game is going to start. In the next step, snake intial moving direction is set along with the speed of the snake which is 20pixel/frame based on the direction snake is moving and it remain same throught the game. After that the snake controlling part started in which the 4 buttons from the Nexys 4 board were used. Direction control assigned based on the position of the button on the board and the middle button is used to restart the game whenever it is pressed. The direction control of the snake is implemented using the if else statements and in these statement collision between snake and wall conditions are also implement. To detect the collision, the horizontal and the vertical pixel value of the snake are check with the horizontal and vertical pixel values of the VGA screen in case if both are equal the GAME_OVER variable become true. Another important feature which is used in these statements are probihition of the movement in the reverse direction means if the player presses the opposite direction button while the snake is moving, the input will be ignored, and the snake will continue moving in the same direction.

In the top module collision function between food and snake and the blocks logic is defined which are stored in two variables. This food collision variable is used in the sanke module to increase the size of the snake based on each successive collision between snake and the food. The increase in size is constant 10 pixels in the horizontal direction and 10 pixels in the vertical direction. If the block collision variable is true, then the GAME_OVER variable wille be true and the game will be over. In the next step, the snake drawing function is implemented which when the snake coordinates align, it draws the snakes on the screen depend on the color defined.

1.3.4 Food module

In this module, operation related to rthe foods was performed. At first, the initial position of the food was initialized which after the first collision between the snake and food initialized randomly. The size of the food is always the same which is 10x10 pixels and the color of the food is also the same which is yellow. In the next step, the random food initialization task was carried out. For this task, a random number generator function was used which store value in the 16-bit array and used the feedback operator as well to make sure the generated

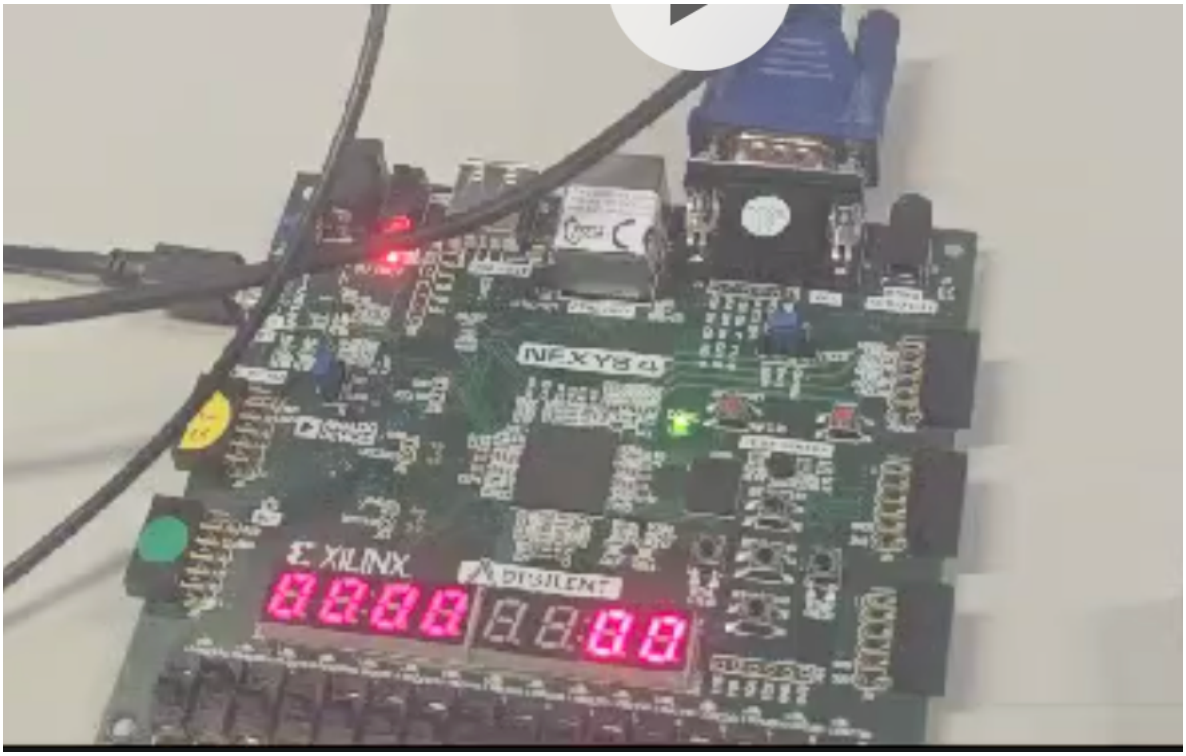
number is always different, and we used module with the VGA screen maximum pixel values in both directions to make sure the food generated inside the display part of the VGA screen. Another important condition implemented using an **if statement** ensures that a new coordinate is generated for the food if its previously generated position overlaps with a block on the screen. In the next step, the food drawing function is implemented which when the food coordinates align, it draws the food on the screen depend on the color defined.

1.3.5 Blocks module

This is a rather simple module. In this module, two blocks were draw on the screen. Ths purpose of these blocks is to make the game more difficult and interesting. The size and the position of these blocks are fixed. One block is at the top middle of the screen and the other block is on the middle bottom of the screen. The color of the blocks is blue. At the end of the module, the block drawing function is implemented which when the block coordinates align, it draws the blocks on the screen depend on the color defined.

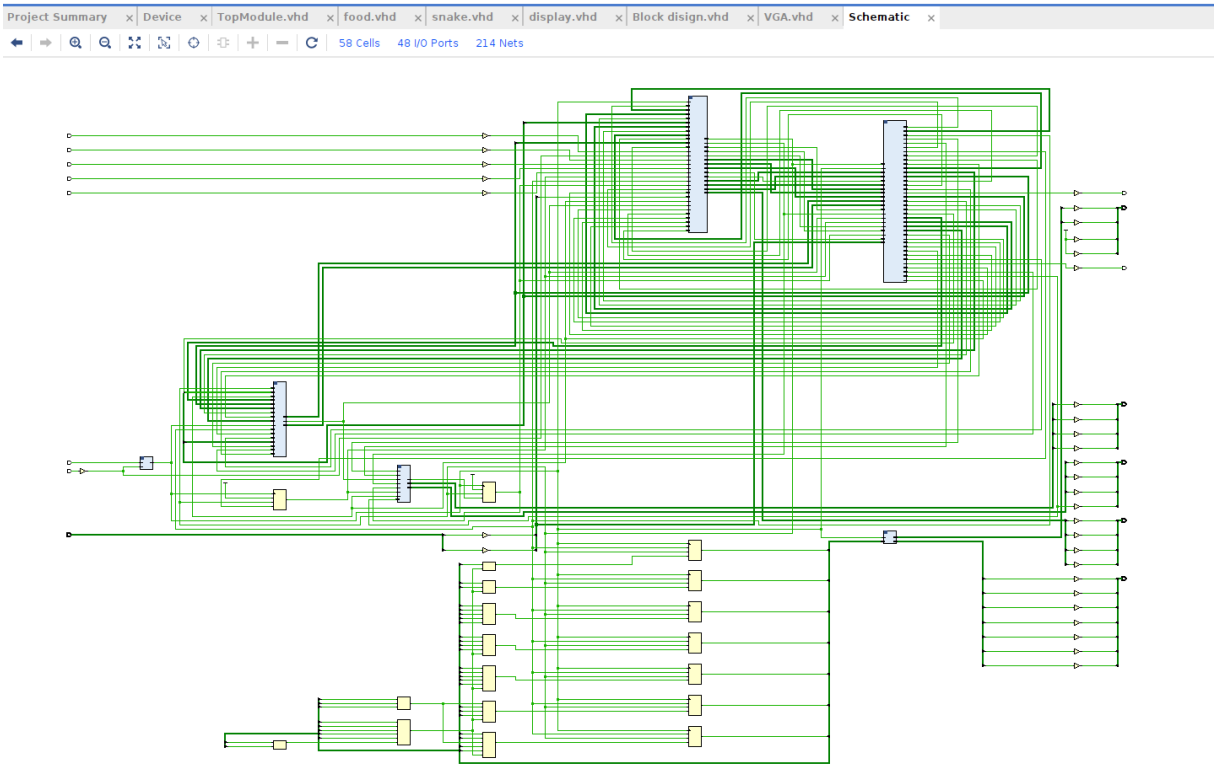
1.3.6 7 Segment Display

To show the current score of the player, two 7 segment displays from the Nexys 4 boards was used. There is counter which increases its value on every successful collision between food and the snake. These displays are controlled using **anode signals (**an**)**, enabling only the required segments while multiplexing between them to maintain a stable visual output.



1.4 Program line & Block Diagrams:

The overall progame lines are **800**. Below is the block diagram of the Snake game:

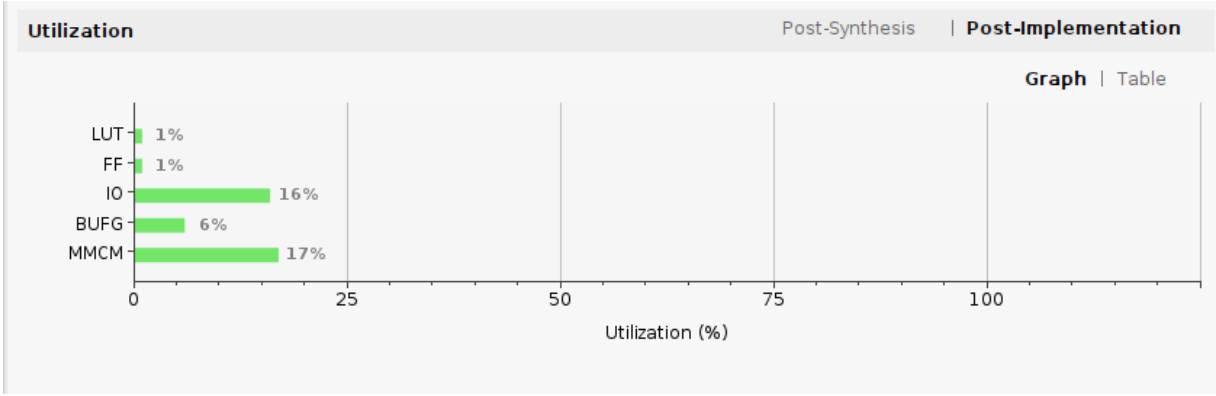


1.4.1 Resources Utilization:

Utilization Post-Synthesis | **Post-Implementation**

Graph | **Table**

Resource	Utilization	Available	Utilization %
LUT	246	63400	0.39
FF	120	126800	0.09
IO	34	210	16.19
BUFG	2	32	6.25
MMCM	1	6	16.67



1.5 Operations:

The resources used from the Nexys 4 board are:

1. 5 Pushbuttons (Left button, Right button, Top button, Down button, Center button)
2. 2 7-Segment displays (Extreme Right)
3. 2 Switches (SW-14, SW-15)
4. Reset button

Push buttons are used to control the movement of the snake. The center button is used to restart the game.

7-Segment display is used to display the current score of the player, and they are incremented by 1 on ebery successful collision.

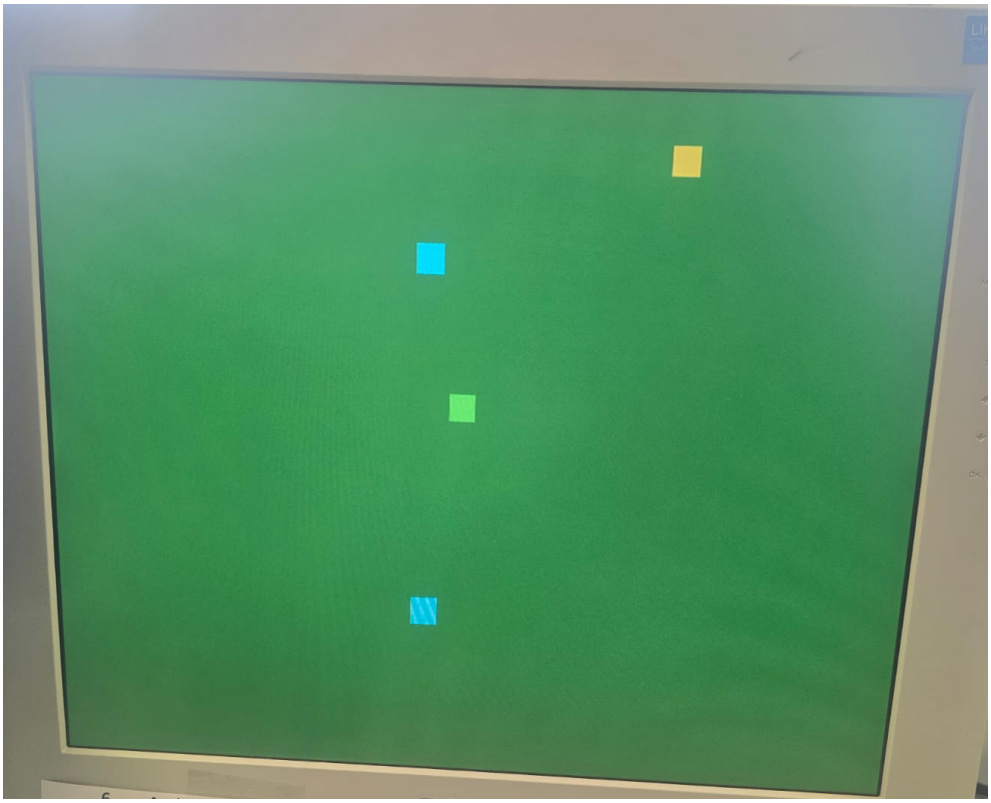
SW-15 is used to pause the VGA screen.

SW-14 is used to disable the functionality of Game over. When its is set to 0 then game will never be over.

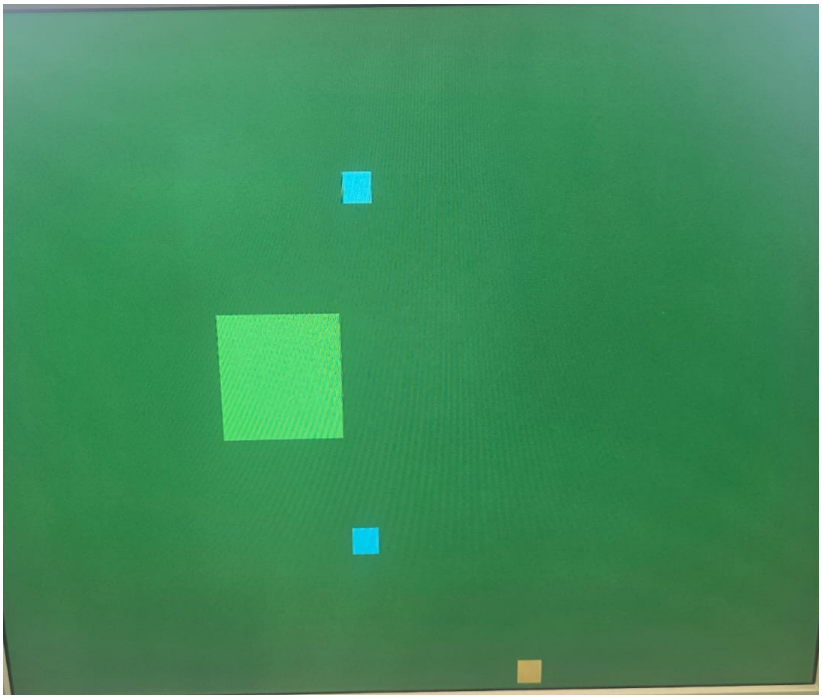
Reset button is used to reset the whole game. One the reset button is pressed the game start from all over again.

1.6 Pictures:

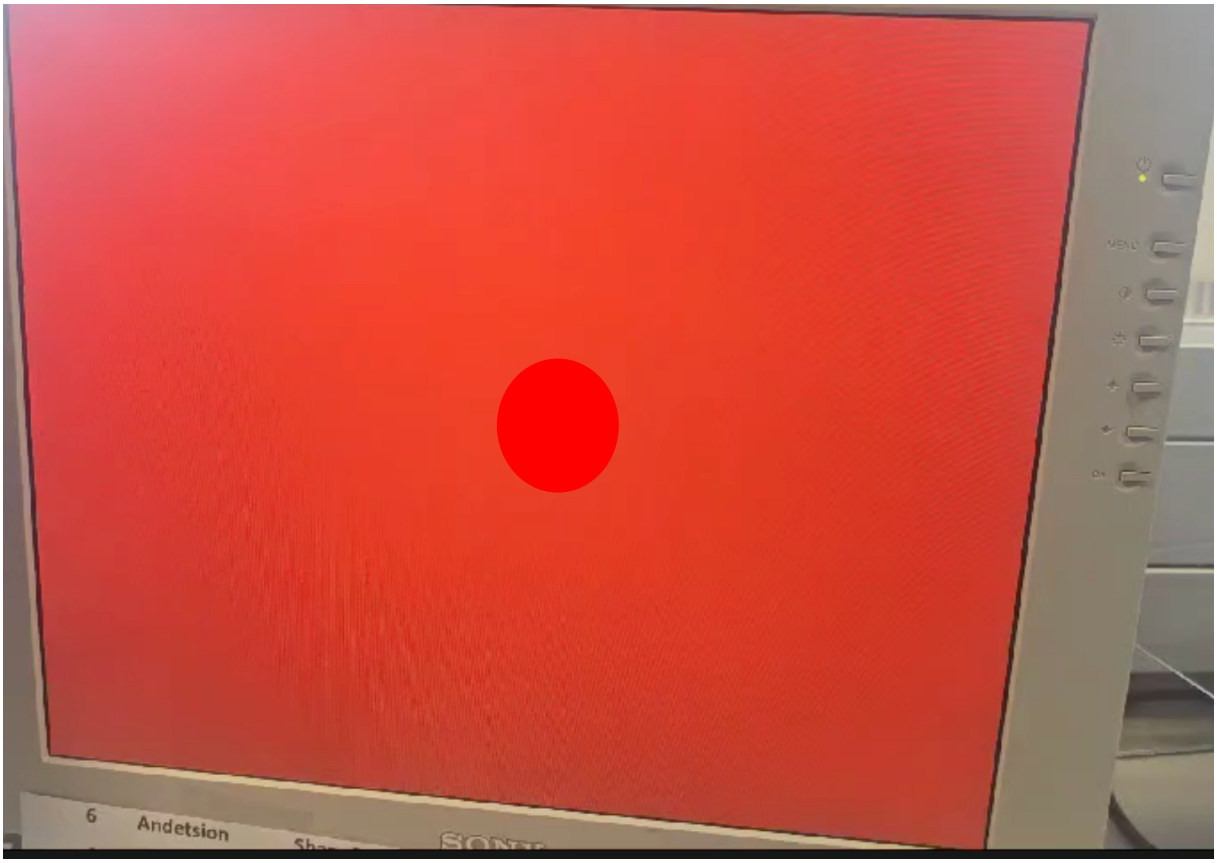
At the Start of the Game, the size of the snake and the food is same as we can see from the picture as well.



As the snake eats the food its size increase in both directions as seen in the below picture:



In case, if snake hit the border of the screen or any of the blue block it dies, and a red screen will appear as shown in the picture. If you press the center button on the Nexys 5 board the game will start again. The below picture shows the Game over screen of the game.



1.7 Conclusion:

This project successfully implements a functional system using the Nexys 4 FPGA board, integrating VGA display, input controls, and a scoring mechanism. Through modular design and efficient logic implementation, key game mechanics such as movement, collision detection, and score tracking were achieved. The project demonstrates the effective use of hardware description languages for real-time interactive applications.